Name: ID:

CS4575/CS6025: Parallel Computing

Winter 2026

Assignment 3: Hybrid MPI + OpenMP Numerical Integration

Weight: 9% of Final Grade

Due: ~3 weeks from release (submit by midnight)

Work Type: Individual

Language: C with MPI and OpenMP

Topic Coverage

- MPI point-to-point communication (MPI_Send, MPI_Recv)
- MPI collective operations (MPI_Reduce)
- OpenMP parallel programming (parallel regions, tasks)
- Hybrid parallel programming (MPI + OpenMP)
- Adaptive algorithms and irregular workloads
- Performance measurement, speedup, and efficiency

Learning Outcomes

- Reuse and extend a parallel MPI program.
- Implement a hybrid MPI + OpenMP solution.
- Understand the difference between global and local parallelism.
- Analyze trade-offs between load balancing and computation speed.
- Evaluate performance of hybrid parallel systems.
- Explain design decisions using concepts from lecture.

2) Problem Overview

You will compute a definite integral of the form:

$$I = \int_0^1 f(x).dx$$

using adaptive numerical integration.

This assignment builds on Assignment 2.

Key idea:

- Assignment 2 used **dynamic MPI scheduling**
- Assignment 3 uses **hybrid parallelism (MPI + OpenMP)**

Important difference:

In this assignment, work is **not redistributed dynamically between processes**.

3. Numerical Method

You must use **Adaptive Simpson's Rule**

Name: ID:

Key idea:

- Compute a Simpson estimate over an interval
- Compute a refined estimate by splitting
- Compare the two estimates
- Accept if within tolerance
- Otherwise, split recursively

4. Functions to Integrate

Your program must support:

ID	Function
0	$f(x)=\sin(x)+0.5\cos(3x)$
1	$f(x)=\frac{1}{1+100(x-0.3)^2}$
2	$f(x)=\sin(200x)e^{-x}$

All integrals are computed over [0,1].

Use the math library (-lm).

5. Program Interface

Command Line Format

mpirun -np P integration func_id mode tol

For hybrid execution:

OMP_NUM_THREADS=T mpirun -np P integration func_id 2 tol

Parameters

- P: number of MPI processes
- func_id: function ID (0, 1, or 2)
- mode: execution mode (see Section 6)
- tol: error tolerance

Example

mpirun -np 8 integration 1 2 1e-8

OMP_NUM_THREADS=4 mpirun -np 4 integration 1 2 1e-8

6. Required Execution Modes

All modes must be implemented in one program.

Mode 0: Serial Baseline

You must reuse your Assignment 2 implementation.

- No parallelism

- Single process performs adaptive integration

Used for:

- correctness verification
- baseline timing

Mode 1: MPI Dynamic Master/Worker (Reuse from Assignment 2)

You must reuse your Assignment 2 implementation.

- Master maintains a task queue
- Workers request work
- Workers split intervals
- Workers send sub-intervals back

This provides:

- dynamic load balancing
- good performance for irregular workloads

Mode 2: Hybrid MPI + OpenMP

This is the core of Assignment 3.

Key Rule

Workers must NOT send sub-intervals back to the master.

Design

Master (Rank 0)

- Split [0,1] into K coarse intervals
- Distribute intervals statically among workers

Workers

- Receive one or more coarse intervals
- Process all intervals locally
- Use OpenMP to parallelize computation

Local Behavior

When an interval is split: [L,R]→[L,M]+[M,R]

- Both sub-intervals are kept locally
- No MPI communication is performed
- OpenMP is used to parallelize recursive computation

Purpose of Hybrid Mode

To study:

- effect of removing dynamic load balancing
- benefit of local parallelism using OpenMP

Name: ID:

7. Correctness Requirements

For each function and tolerance:

$|I_{parallel} - I_{serial}| \leq 10 \times tol$

8. Performance Experiments

Required configurations

MPI (Mode 1)

- Processes: {1, 2, 4, 8, 16}

Hybrid (Mode 2)

- At least 3 combinations of (P, T) - P is the number of MPI processes and T is the umber of OpenMP threads (per process)
    - Example:
        - (2,4)
        - (4,2)
        - (4,4)

Tolerances

- {1e-6, 1e-8}

Functions

- At least two (including function 1 or 2)

Timing rules

- Use MPI_Wtime()
- Run each configuration 10 times
- Discard the first run
- Report average

9. Required Metrics

Runtime

Speedup

$S = T_{serial}/T_{parallel}$

Efficiency

E = S/#workers

Where:

- MPI → workers = processes
- Hybrid → workers = processes × threads

10. Migration Guide (From Assignment 2)

This assignment is designed to reuse your previous work.

Name: ID:

Step 1: Copy your Assignment 2 code

Include:

- serial implementation
- MPI dynamic implementation

Step 2: Keep Modes 0 and 1 unchanged

- Do NOT redesign MPI dynamic logic

Step 3: Add OpenMP support

Add:

#include <omp.h>

Compile with:

mpicc -fopenmp integration.c -lm -o integration

Step 4: Implement Hybrid Mode

Key changes:

Remove:

- task sending back to master
- global dynamic queue

Add:

- static distribution from master
- local computation using OpenMP

Step 5: Use OpenMP inside workers

Recommended:

- OpenMP tasks for recursion

Step 6: Test incrementally

1. Serial works
2. MPI works (same as A2)
3. Hybrid works with 1 thread
4. Hybrid works with multiple threads

11. Written Report (PDF, 4–6 pages recommended)

Your report must include:

A) Design Summary

- Description of adaptive integration
- MPI dynamic design
- Hybrid design
- OpenMP strategy

Name: ID:

B) Results Tables

- Runtime tables
- Speedup and efficiency

C) Plots

- Speedup vs processes
- Efficiency vs processes

D) Discussion

Answer concisely:

- Why does MPI dynamic perform well for irregular workloads?
- Why might hybrid perform worse?
- Does OpenMP compensate for lack of MPI load balancing?
- When does hybrid outperform MPI?
- What is the impact of thread count and process count?

E) Authorship trail

12. Submission Package

Submit a ZIP file:

A3_LastName_FirstName.zip

```
├── integration.c
├── report.pdf
├── README.md
└── Makefile
```

README must include:

- compile command
- example run commands
- machine specifications
- MPI implementation used

13. Academic Integrity

- All work must be your own
- You must be able to explain your code
- Keep an authorship trail and include it in your report.